



HUS
VNU UNIVERSITY OF SCIENCE

Bài 1: Các khái niệm cơ bản

PGS.TS. Lê Trọng Vĩnh
Khoa Toán – Cơ – Tin học, Trường ĐHKHTN

1. Thuật toán và vai trò của thuật toán
2. Đánh giá thời gian thực hiện thuật toán
3. Ký hiệu O lớn và các vô cùng lớn
4. Các xác định thời gian thực hiện thuật toán
5. Vấn đề mẫu dữ liệu vào
6. Phân lớp bài toán dựa trên độ phức tạp

1. Thuật toán và vai trò

- Khái niệm bài toán và thuật toán:
 - **Bài toán:** Trong KHMT, bài toán là một công việc, một vấn đề nào đó cần giải quyết với sự trợ giúp của máy tính. Thuật ngữ “bài toán” được hiểu là một lớp các bài toán cùng loại (khác nhau về tham số).
 - Bài toán: Thông tin cho trước (input) – đầu vào, thông tin cần tìm (output)- đầu ra.
 - **Thuật toán:** Quá trình giải bài toán là sử dụng **một dãy hữu hạn các thao tác đơn giản** được sắp xếp theo một trình tự xác định để biến đổi từ thông tin đầu vào thành thông tin đầu ra. Quá trình này được gọi là thuật toán hay giải thuật của bài toán.
- Biểu diễn thuật toán
 - Dùng ngôn ngữ tự nhiên.
 - Dùng mã giả (pseudo code).
 - Dùng sơ đồ khối (lưu đồ).

1. Thuật toán và vai trò

- Các đặc trưng của thuật toán:
 - **Tính dừng (kết thúc):** Thuật toán phải có tính dừng nghĩa là phải đưa ra được output từ input sau hữu hạn bước.
 - **Tính xác định (đơn nghĩa):** Ở mỗi bước trong thuật toán, các thao tác phải hoàn toàn xác định, đơn trị và không có sự nhập nhằng. Nói cách khác, sau khi thực hiện một thao tác thì hoặc thuật toán kết thúc hoặc có đúng một thao tác được thực hiện. Điều này đảm bảo rằng từ cùng một thông tin vào thì cùng đưa đến một thông tin ra.
 - **Tính phổ dụng:** Thuật toán chạy được với mọi bộ tham số của bài toán
 - **Tính hiệu quả:**
 - Thời gian: Phải nằm trong giới hạn cho phép
 - Không gian: Dung lượng để lưu input, output và các kết quả trung gian nằm trong giới hạn cho phép
 - Khả thi: Phải cài đặt được

1. Thuật toán và vai trò

- **Đánh giá thuật toán:**
 - **Giải một bài toán:** Có nhiều cách khác nhau -> cần chọn thuật toán tốt nhất theo một nghĩa nào đó.
 - **Tính hiệu quả:** Tiết kiệm về không gian và thời gian. Trong hai tiêu chí này thì tiêu chí về thời gian quan trọng hơn.
 - Ví dụ: Tính định thức cấp n của một ma trận vuông $n \times n$: $A = a_{ij}$ theo đệ qui và phương pháp khử Gauss-Jordan
 - Thuật toán đệ qui: $n!$
 - Phương pháp khử Gauss-Jordan: n^3
 - Xem thử với $n=100$?

2. Đánh giá thời gian...

- Tính độc lập:
 - Không phụ thuộc vào phần cứng máy tính, ngôn ngữ lập trình, chương trình dịch
 - Không cần phải lập trình
 - Chỉ cần phân tích thuật toán.
- Các phép toán sơ cấp trong mô hình máy tính truy cập ngẫu nhiên (RAM):
 - Các phép tính số học
 - Các phép tính logic
 - Phép gán, các phép toán truy nhập bộ nhớ (đọc, ghi, sao chép)
 - Các câu lệnh đơn, câu lệnh điều khiển, gọi thủ tục con,...

2. Đánh giá thời gian...

- Kích thước dữ liệu vào
 - Thời gian thực hiện thuật toán: là tổng số các phép toán sơ cấp để biến đổi từ input thành output
 - Kích thước dữ liệu vào và mẫu dữ liệu ảnh hưởng đến số phép toán?
 - Kích thước dữ liệu vào và mẫu dữ liệu chỉ biết được khi thực hiện!
 - Người ta thường dùng một số nguyên dương n để ký hiệu cho kích thước dữ liệu.
 - Một thuật toán T áp dụng để giải một bài toán kích thước n sẽ cần một tổng số $T(n)$ các phép toán sơ cấp. $T(n)$ là độ phức tạp của thuật toán và là đặc trưng cho tính hiệu của của thuật toán.

3. Ký hiệu O lớn và...

- Tốc độ tăng của các hàm
 - Giả sử T_1 và T_2 là hai thuật toán để giải cùng một bài toán kích thước n và thời gian thực hiện tương ứng là:
 - $T_1(n) = C_1n$
 - $T_2(n) = C_2n^2$
 - Trong đó, C_1 và C_2 là hằng số. Rõ ràng khi n đủ lớn thì $T_2 \gg T_1$.
 - Trong các lĩnh vực vô cùng lớn, các hằng số không quan trọng. Độ lớn phụ thuộc chủ yếu vào sự tăng các đối số (ở đây là n). $T(n)$ tăng phụ thuộc vào sự tăng của n .
 - Ký hiệu O lớn được đặt ra nhằm mục đích loại bỏ các thành phần không quan trọng, làm rõ tốc độ tăng của các hàm khi đối số tăng.

3. Ký hiệu O lớn và...

- Ký hiệu O lớn

- **Định nghĩa:** Giả sử $f(n)$, $g(n)$ là hai hàm đồng biến không âm theo n . Ta nói $f(n)$ là O lớn của $g(n)$ và viết $f(n) = O(g(n))$ khi và chỉ khi tồn tại một hằng số C để $f(n) \leq C \cdot g(n)$ kể từ $n \geq n_0$ nào đó.
- **Ví dụ:** $f(n) = 2n^2 + 3n + 5 \Rightarrow f(n) \leq 2n^2 + 3n^2 + 5n^2 = 12n^2$ với mọi $n \geq 1$. Ta viết $f(n) = O(n^2)$.
- **Ý nghĩa:** Ta viết $f(n) = O(g(n))$ nghĩa là tốc độ tăng của $f(n)$ khi n tiến tới vô cùng không vượt quá tốc độ tăng của $g(n)$. Khi n lớn, $g(n)$ cho ta hình dung được mức độ lớn của $f(n)$. Hay nói cách khác, $g(n)$ là thước đo độ lớn của $f(n)$.
- Thông thường, người ta cố gắng ước lượng $g(n)$ sao cho sát nhất với $f(n)$.

3. Ký hiệu O lớn và...

- Tên gọi của các vô cùng lớn

Ký hiệu O lớn	Tên gọi
$O(1)$	Hằng số
$O(\log n)$	Logarit
$O(n)$	Tuyến tính
$O(n \log n)$	Gần tuyến tính
$O(n^k)$	Đa thức
$O(2^n)$	Hàm mũ
$O(n!)$	Giai thừa

3. Ký hiệu O lớn và...

- Thời gian thực hiện thuật toán

Log N	N	$N \log N$	N^2	N^3	2^N
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	54	256	4096	65536
5	32	160	1024	32768	4,294,967,296

4. Tính thời gian...

- Quy tắc tổng và nhân
 - Giả sử $T_1(n) = O(f(n))$ và $T_2(n) = O(g(n))$ tương ứng là độ phức tạp của hai thủ tục P_1 và P_2 .
 - Nếu P là thủ tục gọi tuần tự P_1, P_2 , khi đó độ phức tạp $T(n)$ của P là: $T(n) = O(f(n)) + O(g(n)) = \max(O(f(n)), O(g(n)))$.
 - Nếu P là thủ tục gọi lồng nhau của $P_1 (P_2)$, khi đó độ phức tạp $T(n)$ của P là: $T(n) = O(f(n)) * O(g(n))$.

4. Tính thời gian...

- Theo mô hình RAM
 - Qui ước: gọi $f(n)$ và $f_i(n)$ là thời gian thực hiện của các lệnh S và S_i tương ứng.
 - Các phép toán sơ cấp: $O(1)$
 - Các câu lệnh hợp thành $S = \{S_1, S_2, \dots, S_k\}$: theo qui tắc tổng
 - Câu lệnh rẽ nhánh $S = \text{if}(\text{expL}) \{S_1\} \text{ else } \{S_2\}$: $f(n) = \max(O(f_1(n)), O(f_2(n)))$
 - Câu lệnh lặp $\text{while}(\text{expL})\{S\}$: số lần lặp $\times f(n)$

5. Mẫu dữ liệu vào

- Với một hàm số cho trước, tùy theo giá trị của đối số hàm số có thể nhận được giá trị max hay min.
- Tùy theo mẫu dữ liệu vào $T(n)$ có thể nhận các giá trị sau:
 - $T(n)$ nhỏ nhất. Ký hiệu $T_{\min}$. Đây được gọi là độ phức tạp trong trường hợp tốt nhất.
 - $T(n)$ lớn nhất. Ký hiệu $T_{\max}$. Đây được gọi là độ phức tạp trong trường hợp xấu nhất.
 - $T(n)$ ngẫu nhiên. Ký hiệu T_{aver} . Đây được gọi là độ phức tạp trong trường hợp trung bình.

6. Phân lớp bài toán

- **Định nghĩa 1:** Độ phức tạp của bài toán là độ phức tạp của thuật toán tốt nhất (có độ phức tạp nhỏ nhất) để giải nó
- **Định nghĩa 2:** Bài toán B được gọi là “dẫn về được” bài toán A nếu có thuật toán đơn định đa thức để giải A thì cũng có thuật toán đơn định đa thức để giải B. Ký hiệu $B \alpha A$.
- **Định nghĩa 3:** Bài toán A được gọi là khó tương đương bài toán B ký hiệu $A \sim B$, nếu $A \alpha B$ và $B \alpha A$

6. Phân lớp bài toán

- **Lớp bài toán P và NP:**
 - **P (polynomial):** là lớp các bài toán giải được bằng thuật toán đơn định trong thời gian đa thức
 - **NP (Non polynomial):** là lớp các bài toán không giải được bằng thuật toán đơn định trong thời gian đa thức
 - **Câu hỏi:** P vs NP
- **Lớp bài toán NP-Hard và NP-Complete:**
 - **Định nghĩa 4:** Bài toán A được gọi là NP-Hard (NP-Khó) nếu với mọi B thuộc NP đều có $B \leq A$.
 - **Định nghĩa 5:** Bài toán A được gọi là NP-complete (NP-Đầy đủ) nếu A thuộc NP và A thuộc NP-Hard.